

TD/TP 7

Informatique 2 : Algorithmique 2 / Python MIP - S2

Exercice 1 :

1. Quelles sont les étapes de preuve par induction
2. Quelles sont les étapes de preuve par invariance de boucle

Exercice 2 :

1. Démontrer la correction des formules mathématique suivantes (par induction)
2. A quoi va nous servir ces démonstrations (dans la programmation)

a. Formule 1

$$\sum_{i=1}^n (2i - 1) = n^2$$

C'est à dire :

$$(2 * 1 - 1) + (2 * 2 - 1) + (2 * 3 - 1) + \dots + (2 * n - 1) = n^2$$

b. Formule 2

$$\sum_{i=1}^n 2^i = 2^{n+1} - 2$$

C'est à dire :

$$2^1 + 2^2 + 2^3 + \dots + 2^n = 2^{n+1} - 2$$

c. Formule 3

$$\sum_{i=1}^n f_i^2 = f_n * f_{n+1}$$

Avec, f est la fonction de Fibonacci : $f_n = f_{n-1} + f_{n-2} = 0 + 1 + 1 + 2 + 3 + 5 + 8 + 13 + \dots$

Exercice 3 :

3. Démontrer la correction des algorithmes suivants (par invariance de boucle)

Tri par sélection

```
def tri_selection(arr):
    n = len(arr)

    for i in range(n - 1):
        # Trouver l'élément minimum dans la partie non triée du tableau
        indice_min = i
        for j in range(i + 1, n):
            if arr[j] < arr[indice_min]:
                indice_min = j

        # Échanger l'élément minimum trouvé avec le premier élément
        arr[i], arr[indice_min] = arr[indice_min], arr[i]

    return arr
```

Tri par insertion

```
def tri_par_insertion(T):
    for i in range(1, len(T)):
        x = T[i]
        j = i
        while j > 0 and x < T[j-1]:
            T[j] = T[j-1]
            j = j - 1
        T[j] = x
```

Exercice 4 :

Vérifier si les différents programmes suivants terminent ou pas :

```
i = 1
while i != 10:
    print(i)
    i += 2
```

```
# i = -1
# i = 20
# i = 21

while i != 0:
    print(i)
    i -= 2
```

```
def countdown(i):
    if i == 0:
        print("Terminé!")
    else:
        print(i)
        countdown(i-2)
```

Avec i est un entier positif ici